R

# RESQORA

## A community emergency response network — when seconds matter, the people already closest to you become the first responders.

Project report and technical documentation. A React Native application with a real-time Node.js backend, built to alert, locate and coordinate help within a 2 km radius.

| 2 km | <100 ms | 140 | ₹0 |
|---|---|---|---|
| ALERT RADIUS | ALERT DELIVERY | AUTOMATED CHECKS | PAID APIS USED |

## 01 The problem, and the idea

**In an emergency, the fastest help is not the official kind — it is the person two streets away who has no idea anything is wrong.**

Calling 100 or 112 starts a process: a call is taken, an address is understood, a vehicle is dispatched, traffic is crossed. That process is measured in tens of minutes. Meanwhile the person who could have reached you in ninety seconds walks past, unaware.

RESQORA closes that gap. One button broadcasts your live location, a message and a photo to every registered user within two kilometres, instantly and simultaneously. They see how far away you are, accept with one tap, and navigate to you on a live map while you watch them approach.

**WHO IT IS FOR**

Anyone who walks home alone, travels at night, or lives somewhere help is slow to arrive.

**WHAT MAKES IT DIFFERENT**

No call centre, no dispatcher, no middle layer. Peer to peer, over a socket, in under a tenth of a second.

**THE HARD CONSTRAINT**

An alert that reaches nobody is a failure. Three separate fallbacks exist for exactly that case.

> **The design principle behind every decision here**
> An emergency feature that only works when conditions are good is not a feature. The phone will be backgrounded, the socket will drop, the network will be poor, nobody may be nearby, and the battery will die. Each of those was treated as the normal case, not the exception.

## 02 How an SOS travels




One button press reaches three different audiences over three different channels.

1. **Hold to arm, then a window to cancel**
   Three seconds of deliberate pressure prevents pocket triggers; a five-second countdown follows, cancellable, during which a photo can be attached. Both gates live on one screen — an emergency should never cost an extra navigation.

2. **The server decides who is near**
   The Haversine formula computes great-circle distance to every located user. Anyone inside the 2 km radius is written into the responder table as `alerted`.

3 **Live sockets first, push second**

Every alerted user with an open socket receives the alert immediately. Only those without one fall back to a push notification, so nobody is alerted twice and nobody is missed.

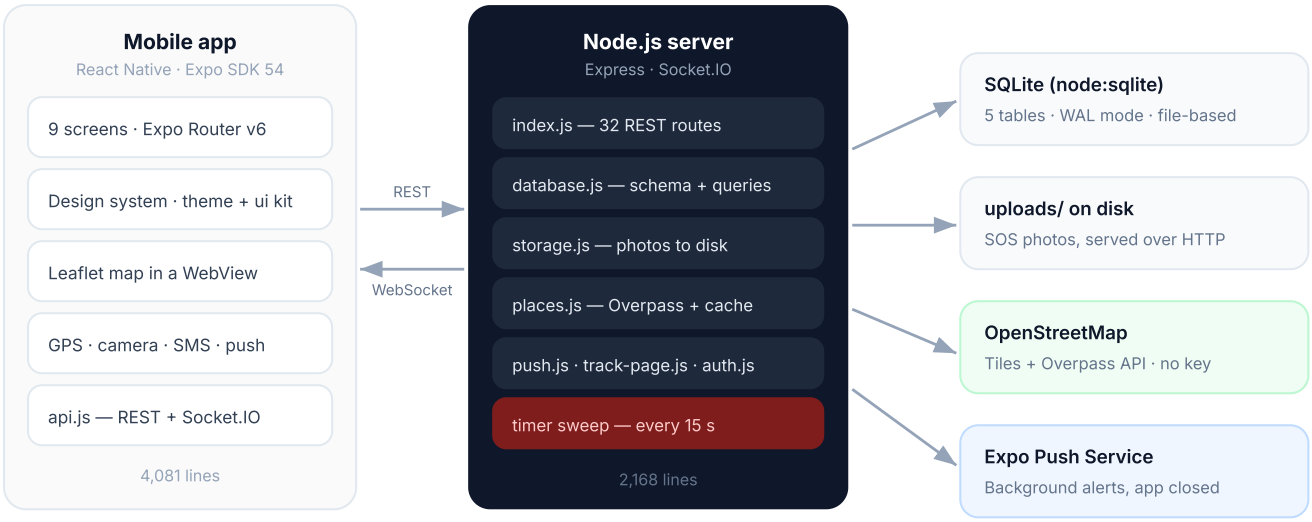
4 **Both sides watch each other move**

The victim's screen plots every responder converging on them; each responder sees the victim, a dashed route line and a live distance that updates as either of them moves.

5 **Arrival, and the end**

A responder marks arrival; the victim's list updates in place. When the victim marks themselves safe, every responder is told at once and location sharing stops in the same instant.

## 03 System architecture




Two processes and one file-based database. Every external dependency is free and replaceable.

### Why this shape

| | |
|---|---|
| **One server process** | REST and WebSocket share memory, so the socket registry and the database are always consistent. No queue, no cache layer, nothing to fall out of sync. |
| **SQLite, not Postgres** | Node 22+ ships `node:sqlite` in the standard library. Zero setup, zero credentials, one file to back up — and comfortably fast for a campus or neighbourhood deployment. |
| **Photos on disk, URLs in the database** | A 400 KB photo never travels through SQLite or a socket frame. Only a short URL does. |
| **ngrok in front** | Gives the laptop a public HTTPS address, so phones work on any network — mobile data included — without deploying anywhere. |

## 04 What the system does

### The core loop

TRIGGER

#### Hold, confirm, send

Two safety gates on one screen. A photo of the scene can be attached before it goes, or added later while help is on the way.

BROADCAST

#### Everyone within 2 km

Location, a default message and the photo, delivered by socket in under 100 ms, or by push notification if the app is closed.

RESPOND

#### Live two-way map

Real OpenStreetMap. The responder sees the victim and a route line; the victim watches each responder close the distance.

RESOLVE

#### Arrival and all-clear

Arrival updates the victim's screen instantly. Marking yourself safe notifies everyone and stops location sharing in the same moment.

### Reaching beyond the network Added in v2

#### Emergency contacts

Up to five trusted people. Raising an SOS opens your messaging app with all of them and the alert prefilled — one tap to send. No SMS gateway, no cost, and the message genuinely comes from your number.

#### Public live-tracking link

Every SOS mints an unguessable URL. It opens a live map of the person's position, message and photo in any browser — no app, no account. It stops updating and says so the moment the emergency ends.

### Protection before anything happens Added in v2

#### Safety timer

“Expect me in 20 minutes.” Miss the check-in and the server raises a full SOS on your behalf, using your last known position. The deadline lives on the server, so it fires even if the phone is taken, switched off or out of battery.

#### Nearest official help

Nearest police stations and hospitals from OpenStreetMap, with distance, one-tap calling and directions — plus a permanent **Call 112** button that works even when the lookup does not.

### Everything else

| | |
|---|---|
| **Accounts** | Phone and password, bcrypt hashing, 30-day JWT, automatic sign-in on reopen. |
| **History** | Every SOS kept permanently — time, duration, message, photo and who came. Grouped by day; an active one is tappable to rejoin it. |
| **Profile** | Stats, alert radius, emergency contacts, and a “clear active SOS” escape hatch. |
| **Withdrawal** | “I can't make it” tells the victim, so they stop counting on someone who is not coming. |
| **Recovery** | Missed alerts are replayed, stuck sessions can be cleared, and forgotten ones auto-resolve after six hours. |

## 05 Technology used

### MOBILE APPLICATION

| | |
|---|---|
| `expo ~54` | Managed React Native toolchain — build, assets, native modules without Xcode or Android Studio. |
| `expo-router 6` | File-based navigation. Every file under `app/` is a route. |
| `socket.io-client` | The real-time channel. Auto-reconnects, which the recovery logic depends on. |
| `react-native-webview` | Hosts the Leaflet map — how a real map arrives with no API key. |
| `expo-location` | GPS: a one-shot fix at trigger time and a distance-filtered watcher during an emergency. |
| `expo-image-picker` | Camera and gallery for the scene photo. |
| `expo-notifications` | Push registration, an emergency-priority Android channel, and tap-to-respond. |
| `expo-sms / expo-clipboard` | Prefilled messages to emergency contacts; copying the tracking link. |
| `expo-av` | The alert siren on an incoming SOS. |
| `@expo/vector-icons` | Ionicons throughout — no emoji standing in for an icon. |
| `@expo-google-fonts/inter` | Inter, with tabular figures so timers and distances do not jitter. |

### SERVER

| | |
|---|---|
| `express` | 32 REST routes, static photo serving, JSON error handling. |
| `socket.io` | Real-time fan-out, with a userId → many-sockets registry for multi-device delivery. |
| `node:sqlite` | Built into Node 22+. Five tables, WAL mode, prepared statements everywhere. |
| `bcryptjs` | Password hashing at cost factor 10. |
| `jsonwebtoken` | Signed 30-day tokens; an optional-auth variant personalises public endpoints. |
| `node:crypto / node:fs` | Unguessable tracking tokens; photo storage on disk. |

### EXTERNAL SERVICES — ALL FREE, NONE REQUIRING A KEY OR A CARD

| | |
|---|---|
| `OpenStreetMap` | Map tiles (CARTO dark style) and place data. |
| `Overpass API` | Nearest police stations and hospitals. Two mirrors, cached an hour. |
| `Expo Push` | Background notifications when the app is closed. |
| `ngrok` | Public HTTPS tunnel to the development machine. |

## 06 Data model

| TABLE | HOLDS |
|---|---|
| **users** | name, phone (unique), bcrypt password, push token, online flag, last position, timestamps |
| **sos_sessions** | victim, live coordinates, message, photo URL + type + name, status, radius, share token, started/ended |
| **sos_responders** | session, responder, status (alerted → responding → arrived, or declined), responded/arrived times — one row per pair, enforced by a unique index |
| **emergency_contacts** | owner, name, phone — capped at five, unique per person |
| **safety_timers** | owner, note, status, expiry, warned flag, and the SOS it raised if it expired |

> **Migrations run themselves**
> The schema is created and patched on boot — new columns are added only if missing, duplicate rows are cleaned before a unique index is applied, and indexes are created idempotently. An existing database upgrades in place with no manual step; this was verified against the real database holding 8 accounts and 40 sessions.

## 07 The API surface

### PUBLIC

| | | |
|---|---|---|
| GET | / | Health check with live counters |
| GET | /config | Default message, radius, image limits, hotlines |
| GET | /stats | Online count (excluding you) and active emergencies |
| POST | /auth/register | Create an account |
| POST | /auth/login | Sign in, receive a 30-day token |
| GET | /t/:token | Public live-tracking page for one emergency |
| GET | /t/:token/data | The live JSON that page polls |
| GET | /uploads/:file | SOS photos |

### AUTHENTICATED — EMERGENCIES

| | | |
|---|---|---|
| POST | /sos/trigger | Raise an SOS: location + message + photo |
| GET | /sos/active | Your own running emergency, for restoring the live screen |
| GET | /sos/incoming | Live alerts near you that still need someone |
| GET | /sos/:id | Full details: victim, message, photo, responders |
| POST | /sos/:id/image | Attach or replace the photo mid-emergency |
| POST | /sos/:id/respond | Accept — “I'm on my way” |

| | | |
|---|---|---|
| POST | /sos/:id/arrived | Mark arrival |
| POST | /sos/:id/withdraw | “I can't make it” — the victim is told |
| POST | /sos/:id/end | All clear |
| POST | /sos/mine/clear | Close anything stuck open |

### AUTHENTICATED — EVERYTHING ELSE

| | | |
|---|---|---|
| GET | /nearby | Online users within range (no coordinates returned) |
| GET | /nearby/help | Nearest police and hospitals + hotlines |
| GET | /timer POST /timer/start | Safety timer state; start one |
| POST | /timer/checkin · /extend · /cancel | “I'm safe”, buy ten more minutes, or call it off |
| GET | /contacts POST DEL | Emergency contacts, max five |
| GET | /history GET /profile/stats | Past emergencies; counters |
| GET | /user/me POST /user/location · /push-token | Profile, position, device token |

## 08 The real-time layer

Fourteen events carry everything that must happen instantly. The server keeps a `userId → Set<socketId>` registry, so a person with two devices gets every event on both.

| EVENT | DIRECTION | MEANING |
|---|---|---|
| `authenticate` | app → server | Bind this socket to a user |
| `location_update` | app → server | New GPS fix; relayed to responders if an SOS is live |
| `responder_location_update` | app → server | A responder's position, forwarded to the victim |
| `sos_alert` | server → nearby | Someone needs help: location, message, photo, distance |
| `sos_image` | server → responders | A photo was added or replaced mid-emergency |
| `responder_coming` | server → victim | Someone accepted |
| `responder_location` | server → victim | They moved; includes recomputed distance |
| `responder_arrived` | server → victim | They are here |
| `responder_left` | server → victim | They withdrew — stop counting on them |
| `victim_location_update` | server → responders | The victim moved |
| `sos_ended` | server → all | Resolved; stop tracking |
| `sos_resume` | server → client | You reconnected and your own SOS is still live |
| `timer_warning` | server → client | Your safety timer expires in two minutes |
| `timer_expired` | server → client | It expired; an SOS went out for you |

> **Sockets are treated as unreliable by design**
> A backgrounded phone suspends its socket, and anything sent meanwhile is gone permanently. So every screen showing live state re-reads it from the server on reconnect and on every return to the foreground. The socket is an optimisation for speed; the database is the source of truth.

## 09 Interface design

The interface was rebuilt on a token system, because an emergency screen has to be readable at a glance, at night, at arm's length, by someone who is frightened.

### One source of truth

`theme.js` holds every colour, type style, spacing step and radius. `components/ui.jsx` holds the shared Card, Button, Pill, Stat, Field, Avatar and EmptyState. Screens compose; they do not restyle.

### Contrast, measured

Body text sits at 18.9:1 against the near-black background, secondary at 7.4:1, and the faintest tier at 4.6:1 — the WCAG floor. The old palette failed this and was corrected.

### Never colour alone

Every status pairs a colour with an icon and a word: “Arrived” green with a check, “On the way” amber with a walking figure. Colour-blind users lose nothing.

### Reachable targets

Every interactive element is at least 48 dp with a visible pressed state, and motion respects the system reduce-motion setting.

| | |
|---|---|
| **Palette** | Near-black `#09090B` ground; alert red `#EF4444` for emergency, green for safe, blue for you, amber for in-progress. Semantic, never decorative. |
| **Type** | Inter at four weights. Body never below 15 pt. Timers and distances use tabular figures so numbers do not shift as they count. |
| **Icons** | Ionicons throughout. Emoji were removed — they render differently on every platform and cannot be themed. |
| **Structure** | Two screens were merged into one and three files deleted. The fastest interface is the one with less in it. |

## 10 Engineering decisions worth defending

### A real map with no API key

Google Maps requires a billing account and a key that leaks in a client build. Instead, Leaflet renders inside a WebView against OpenStreetMap raster tiles: a genuine, pannable, zoomable map with live markers and a route line — no key, no card, no quota. It works identically in Expo Go and in a standalone build, and falls back to plain coordinates when tiles cannot load.

### The safety timer runs on the server, not the phone

A countdown in the app dies with the app. The deadline is a database row swept every fifteen seconds, so the SOS fires even if the phone is smashed, stolen or flat. This is the difference between a feature and a promise.

### Photos as files, not database rows

The original approach stored base64 images directly in SQLite and pushed them through socket frames — a 400 KB photo becoming a 540 KB string sent to every recipient. Now the server decodes once, validates type and size, writes to disk and stores a short URL. The database stays small and the socket payload stays tiny.

### One code path for automatic and manual alerts

The timer and the SOS button both call the same `raiseSOS()` function. An automatic alert is byte-for-byte identical to a pressed one — there is no second implementation to drift out of sync.

### The online count excludes you

Standing alone and reading “1 online” is worse than useless. The figure is computed per viewer, both in the REST response and in the socket broadcast, so it always means *other people*.

### Location is never exposed casually

The nearby-users endpoint returns names and distances but never coordinates. The public tracking page shows the emergency and nothing about the account — a test asserts the phone number cannot appear in that response. Sharing stops the instant the SOS ends.

### Third parties fail soft

Overpass is a free community service that will be down when it is most needed. Lookups are cached for an hour on a one-kilometre grid, tried against two mirrors, and fall back to stale data and then to a hotline button that depends on nothing.

## 11 Defects found and fixed

Most were the same root cause wearing different clothes: the app trusted a one-shot event and never re-checked.

| DEFECT | CONSEQUENCE AND FIX |
|---|---|
| **Reconnect unregistered the live socket** | A user's socket map was one-to-one, so a dying socket removed the fresh one and the user silently stopped receiving alerts. Now a set of sockets per user. |

| | |
|---|---|
| **Missed alerts were never replayed** | A push notification arrived, the app opened, and showed nothing — the socket event had fired while it was closed. Added a pending-alerts endpoint plus a replay on reconnect. |
| **“I've arrived” never reached the victim** | The database recorded it correctly, but the victim's backgrounded phone missed the event and froze on “On the way”. Both live screens now re-sync on foreground. |
| **“I can't make it” told nobody** | It only navigated away, leaving the victim expecting someone who was not coming. Added a withdraw endpoint and a notification. |
| **Back stranded you outside your own SOS** | One stray tap and there was no way back in. Back is now guarded, History rows are tappable, and returning to Home restores the live screen. |
| **A stuck SOS blocked all future ones** | “You already have an active SOS” with nothing visible anywhere. Added a clear action in Profile, a recovery dialog at trigger time, and a six-hour auto-resolve. |
| **Stale state survived a crash** | Online flags and abandoned sessions persisted across restarts. Both are cleaned at boot. |
| **Errors returned HTML** | An oversized upload produced an Express error page the app could not parse. All failures are JSON now. |
| **Distance froze mid-response** | A stale closure meant the responder's distance stopped updating when the victim moved. |
| **A patch upgrade broke the build** | Expo 54.0.37 ships a polyfill with an untranspiled class that the bundled Hermes compiler rejects, breaking every production build. Versions are pinned deliberately, with the reason recorded. |

## 12 Testing

140 automated checks run against a real server on a throwaway database with one command: `npm test`.

115 CHECKS

### End-to-end suite

Boots the server, registers real accounts, opens real sockets, and drives complete journeys — trigger, alert, respond, arrive, withdraw, resolve.

25 CHECKS

### Contract suite

Asserts that every field each screen reads actually exists in the server's response, so the interface and the API cannot drift apart unnoticed.

#### WHAT THE SUITES PROVE

| | |
|---|---|
| **The radius is real** | A user 0.9 km away is alerted; one at 5.5 km is not; you are never alerted to your own emergency. |
| **Photos survive the round trip** | Stored as a file, downloadable over HTTP, replaceable mid-emergency, and still present in history afterwards. |
| **Missed alerts are recoverable** | A responder is disconnected before an SOS fires, then reconnects and receives it both by REST and by socket replay. |
| **The timer genuinely fires** | The suite backdates a deadline, waits for the real sweep, and confirms a full SOS appeared and reached a nearby responder's socket. |
| **Nothing leaks** | Access control on every route; the tracking page cannot expose an account. |
| **Bad input is handled** | Oversized images, non-image data, malformed coordinates, invalid tokens, duplicate accounts — all rejected as JSON with the right status. |

> **Tests that earned their keep**
> Two failures during development were genuine test bugs rather than product bugs — a socket the suite had deliberately disconnected, and a variable declared twice. Both were investigated rather than silenced, which is exactly what a suite is for. Every fix in Section 11 was locked in with a regression check before moving on.

## 13 Running the system

One command starts everything — database, backend, public tunnel and the app:

| | |
|---|---|
| `.\start.ps1` | Clears ports, installs anything missing, starts the backend (which opens the database), raises the ngrok tunnel, launches Expo — verifying each responded before continuing |
| `.\start.ps1 -Stop` | Shuts all of it down |
| `.\start.ps1 -NoTunnel` | Same-WiFi testing without ngrok |
| `npm test` | All 140 checks, from `server/` |
| `node reset-password.js` | List accounts, or set a password |

The script also warns if the tunnel URL stops matching the one compiled into the app — historically the single most common cause of “the app cannot connect”.

## 14 What comes next

| | |
|---|---|
| **Cloud deployment** | Move off the laptop to a small VPS with a domain and a certificate, replacing the ngrok tunnel. |
| **Push on iOS** | Android delivery works today; iOS needs an Apple Developer account for APNs. |
| **Turn-by-turn inside the app** | Currently hands off to the phone's maps app; OSRM could route in-app for free. |
| **Verified responders** | Optional identity checks, so a victim knows who is arriving. |
| **Abuse prevention** | Rate limiting and false-alarm handling before any public deployment. |

## 15 The project in numbers

| | |
|---|---|
| Application code | **4,081 lines** — 9 screens, 7 shared components, 6 service modules |
| Server code | **2,168 lines** across 8 modules, plus 3 test files |
| REST endpoints | **32** |
| Real-time events | **14** |
| Database tables | **5**, plus photo files on disk |
| Automated checks | **140**, all passing |
| Paid services | **None.** No maps key, no SMS gateway, no cloud bill |
| Alert radius / delivery | **2 km** · under **100 ms** to a connected device |

> **If there is one thing to take away**
> The hard part of an emergency application is not sending the alert. It is guaranteeing that the alert still arrives when the phone is asleep, the socket is dead, the network is poor, nobody is nearby, or the battery is gone — and that the person who needs help can always find their way back to the screen that matters.

RESQORA — community emergency response. Built with React Native, Node.js, Socket.IO, SQLite and OpenStreetMap. Map data © OpenStreetMap contributors.